

QuickLabour - Platform Specifications & Architecture

This document provides a comprehensive technical overview, operational workflows, security architecture, and database schemas of the **QuickLabour** platform—a premium, on-demand marketplace connecting blue-collar trade workers (plumbers, electricians, carpenters, etc.) with clients needing local repair services in real-time.

1. Executive Summary & Core Concept

QuickLabour addresses the fragmentation of on-demand blue-collar labor matching by establishing a real-time marketplace. It caters to three primary roles: 1. **Clients**: Individuals who need swift, trustworthy domestic repair work completed. 2. **Workers (Labours)**: Skilled service providers seeking local job requests matching their specific trade. 3. **Administrators**: Support and operational staff who monitor transactions, moderate reviews, verify credentials, and audit system security.

■■ **NOTE:** > The platform uses a hybrid matching system: once a client posts a job, the backend dynamically queries matching active workers near the location and routes bids or invitations straight to their dashboards.

2. Project Requirements

■ Client Features

- **Authentication & Profiles**: Register with secure passwords; maintain profiles with full names, phone numbers, and addresses.
- **On-Demand Posting**: Post job requests detailing task category (Repair Type), geographic location, detailed address, GPS coordinates, and budgeted price.
- **Match & Bidding Queue**: View nearby matched workers' details, including ratings and completed job counts.
- **Direct Coordination**: Accept/hire a worker. Upon acceptance, the system automatically launches an integrated chat session.

- **Feedback System:** Rate hired workers and submit reviews detailing service quality.

■ Worker (Labour) Features

- **Profile Verification:** Upload identity files (such as Aadhaar cards) for administrator verification.
- **Availability Toggle:** Switch status between "Online & Available" and "Offline" to control the receipt of job alerts.
- **Dashboard Notifications:** View real-time service invitations matching their trade specialty.
- **Navigation Utilities:** Access client locations using integrated coordinates mapped to Google Maps.
- **Interactive Messaging:** Send and receive direct chat messages with hiring clients to coordinate visits and payments.

■■ Administrative Features

- **Performance Overview:** Track total platform metrics including Total Budget Transaction Volume, Matchmaking Rate, and support tickets.
- **User Management Directories:** Search and manage Client, Worker, and Administrator profiles.
- **Verification Audits:** View uploaded physical identity documents (e.g. Aadhaar cards) in an in-browser document viewer to verify worker profiles.
- **Platform Moderation:** Delete active listings, delete fraudulent accounts, delete spam support inquiries, and moderate low-rating reviews.
- **Administrative Registry:** Register new authorized administrators under strict password criteria.
- **Audit Security Console:** A secure administrative credentials auditor enabling superusers to inspect and copy database credential records (emails, Bcrypt hashes) for troubleshooting and integrity analysis.

3. User & System Flows

■ Registration & Onboarding Flow

```
flowchart TD
    A[Start Registration] --> B{Select Role}
```

```
B -->|Client| C[Input Name, Email, Phone, Address, Strong Password]
B -->|Worker| D[Input Name, Email, Phone, Address, Trade, Aadhaar ID]
C --> E[Submit Client Registration]
D --> F[Submit Worker Registration]
E --> G[Session Token Generated via JWT]
F --> G
G --> H[Access Role-Specific Dashboard]
```

■ Job Posting & Auto-Chat Execution Flow

```
sequenceDiagram
    autonumber
    actor Client
    actor Worker
    participant Server as Express Server
    participant DB as MongoDB Atlas

    Client->>Server: POST /api/jobs (Details & Budget)
    activate Server
    Server->>DB: Query Workers (role: "worker", specialty matches job type)
    DB-->>Server: Return workers list
    Server->>DB: Save Job (status: "Waiting...", pre-populated bidders)
    Server-->>Client: Return Job with matching bids
    deactivate Server

    Worker->>Server: PUT /api/jobs/:id/status (Set "Accepted")
    activate Server
    Server->>DB: Update status to "Accepted" & assign hiredWorker
    Server->>DB: Create automatic coordination Message (Worker Details)
    Server->>DB: Create automatic coordination Message (Client Details & GPS Link)
    Server-->>Worker: Status accepted, message thread initialized
    Server-->>Client: Alert: Hired Worker Matched!
    deactivate Server
```

4. Technical Requirements

■ Technology Stack

- **Frontend Layer:** React.js (Vite bundle), React Router Dom (v6+), Vanilla CSS (premium design tokens, custom glassmorphism components), Bootstrap Icons.
- **Backend Layer:** Node.js, Express.js API Gateway, Mongoose ODM.
- **Database Layer:** MongoDB Atlas Cluster holding separate, unified collections.
- **Session Storage:** Client-side state managed via `sessionStorage` containing JWT tokens, user metadata, and active role profiles.

■ ■ Core Architecture Elements

- **Unified User Adapter Pattern:** A custom database facade in `User.js` intercepts model queries and dynamically distributes lookups across independent `Client`, `Labour`, and `Admin` physical collections, achieving logical polymorphism:

```
javascript const User = { findOne: async (query) => { let result = await Client.findOne(query); if (result) return result; result = await Labour.findOne(query); if (result) return result; return await Admin.findOne(query); } }
```
- **Authentication Interceptor:** All secure endpoints are guarded by `authMiddleware.js` which parses JWT Bearer tokens to identify requests, strip secure fields, and bind matching accounts to incoming query requests.

5. Security Architecture

■ ■ **IMPORTANT:** > Security is a central pillar of the QuickLabour system, utilizing modern standard procedures to safeguard both domestic client addresses and physical workers' identity proofs.

1. **Password Hashing:** All user groups (Clients, Workers, and Admins) have passwords cryptographically secured using **BcryptJS** hashing with a work factor (salt rounds) of **10**. Cleartext passwords are never persisted to MongoDB.
 2. **Access Tokenization (JWT):** Session management is secured via **JSON Web Tokens (JWT)** generated at register/login, expiring after **30 days**. Tokens are transmitted via standard `Authorization: Bearer <TOKEN>` HTTP headers.
 3. **Password Complexity Regulation:** Strict password constraints are validated on both client and server layers using standard Regular Expressions: **regex**

```
/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&#])[A-Za-z\d@$!%*?&#]{8,}$/
```

Min 8 chars, 1 uppercase, 1 lowercase, 1 number, and 1 special symbol.
 4. **Security Auditor Console:** The admin portal includes a dedicated secure modal context for credential auditing, allowing Superusers to review hashed security credentials, emails, and full system metadata in a clipboard-ready panel to support developer debugging.
-

6. Backend Database Schema

■ Entity-Relationship Diagram

```
erDiagram
    CLIENT ||--o{ JOB : posts
    LABOUR ||--o{ JOB : bids_on
    LABOUR ||--o{ JOB : hired_for
    CLIENT ||--o{ MESSAGE : sends_receives
    LABOUR ||--o{ MESSAGE : sends_receives
    ADMIN ||--o{ MESSAGE : sends_receives
    REVIEW ||--|| LABOUR : reviews
    CONTACT ||--|| ADMIN : manages
```

■■ Collection Schemas

1. Client Collection (**client.js**)

Stores authenticated clients posting job requests.

Field Name	Data Type	Rules / Validations	Default Value
_id	ObjectId	Auto-generated primary key	-
fullName	String	Required	-
email	String	Required, Unique, Email Match	-
password	String	Required, Bcrypt Hashed	-
phone	String	Required	-
address	String	Required	-
role	String	Constant	"client"

avatar	String	URL String	Default Unsplash avatar
idType	String	Identity Class	"Aadhaar"
idFile	String	Base64 URL	""
rating	Number	Floating rating	5.0
jobsCompleted	Number	Counter	0
createdAt	Date	Timestamp	Auto-generated
updatedAt	Date	Timestamp	Auto-generated

2. Labour Collection (Labour.js)

Stores certified blue-collar workers.

Field Name	Data Type	Rules / Validations	Default Value
_id	ObjectId	Auto-generated primary key	-
fullName	String	Required	-
email	String	Required, Unique	-
password	String	Required, Bcrypt Hashed	-
phone	String	Required	-

address	String	Required	-
role	String	Constant	"worker"
occupation	String	Required (e.g. "Plumber")	-
avatar	String	URL String	Default Unsplash avatar
idType	String	Identity Class	"Aadhaar"
idFile	String	Base64 URL	""
isOnline	Boolean	Availability toggle	true
rating	Number	Floating rating	4.9
jobsCompleted	Number	Counter	0
skills	[String]	Array of keywords	[occupation]
createdAt	Date	Timestamp	Auto-generated
updatedAt	Date	Timestamp	Auto-generated

3. Job Collection (Job.js)

Coordinates repair requests, matching, and active matches.

Field Name	Data Type	Rules / Validations	Default Value
------------	-----------	---------------------	---------------

<code>_id</code>	ObjectId	Auto-generated primary key	-
<code>client</code>	ObjectId	Reference to <code>Client</code> Collection	<i>Required</i>
<code>name</code>	String	Required client descriptor	-
<code>title</code>	String	Generated: <code>[Repair] Request at [Location]</code>	-
<code>location</code>	String	Generic location label	-
<code>fullAddress</code>	String	Detailed residential street address	<code>""</code>
<code>latitude</code>	Number	GPS Float	<code>null</code>
<code>longitude</code>	Number	GPS Float	<code>null</code>
<code>repair</code>	String	Category of repair	-
<code>money</code>	Number	Budget cost in INR	-
<code>status</code>	String	Enum: <code>['Waiting...', 'Accepted', 'Completed', 'Rejected']</code>	<code>"Waiting..."</code>
<code>hiredWorker</code>	ObjectId	Reference to <code>Labour</code> Collection	<code>null</code>

<code>bidders</code>	Array	Nested bidder rate listings	<code>[]</code>
<code>bidders[i].worker</code>	ObjectId	Reference to <code>Labour</code> Collection	-
<code>bidders[i].rate</code>	String	Bid rate string (e.g. <code>"\$550/day"</code>)	-
<code>createdAt</code>	Date	Timestamp	<i>Auto-generated</i>

4. Message Collection (`Message.js`)

Enables direct coordination and chat.

Field Name	Data Type	Rules / Validations	Default Value
<code>_id</code>	ObjectId	Auto-generated primary key	-
<code>senderId</code>	String	Account ID	-
<code>senderName</code>	String	Display name	-
<code>senderRole</code>	String	Enum: <code>['client', 'worker', 'admin']</code>	-
<code>senderAvatar</code>	String	URL String	<code>""</code>
<code>receiverId</code>	String	Account ID	-
<code>receiverName</code>	String	Display name	-

<code>text</code>	String	Max length 1000 characters	-
<code>isRead</code>	Boolean	Message status indicator	<code>false</code>
<code>createdAt</code>	Date	Timestamp	<i>Auto-generated</i>

5. Review Collection (`Review.js`)

Collects general landing page reviews.

Field Name	Data Type	Rules / Validations	Default Value
<code>_id</code>	ObjectId	Auto-generated primary key	-
<code>name</code>	String	Author name	-
<code>sub</code>	String	Author subtitle/occupation	-
<code>text</code>	String	Review content	-
<code>avatar</code>	String	URL	<i>Default portrait URL</i>
<code>rating</code>	Number	Min 1, Max 5	-
<code>workerType</code>	String	Associated specialty tag	-
<code>createdAt</code>	Date	Timestamp	<i>Auto-generated</i>

6. Contact Collection (**Contact.js**)

Holds active customer support entries.

Field Name	Data Type	Rules / Validations	Default Value
_id	ObjectId	Auto-generated primary key	-
name	String	Contact name	-
email	String	Contact email	-
subject	String	Subject label	-
message	String	Detailed support description	-
createdAt	Date	Timestamp	<i>Auto-generated</i>

7. Admin Collection (**Admin.js**)

Details platform control center superusers.

Field Name	Data Type	Rules / Validations	Default Value
_id	ObjectId	Auto-generated primary key	-
fullName	String	Required	-
email	String	Required, Unique	-
password	String	Required, Bcrypt Hashed	-

phone	String	Required	-
avatar	String	URL	""
role	String	Constant	"admin"
createdAt	Date	Timestamp	<i>Auto-generated</i>